

jelaskan konsep yang biasanya baru dipahami setelah bekerja di data warehouse, seperti:

Slowly Changing Dimension (SCD)

Surrogate Key vs Business Key

Fact Table Grain

Snowflake vs Star Schema

Incremental vs Full Load

Partitioning Fact Table

Ini biasanya materi senior data engineer / BI engineer, tapi sangat membantu memahami ETL secara mendalam.

Baik. Ini memang konsep yang biasanya **baru benar benar dipahami setelah beberapa tahun bekerja dengan data warehouse**. Waktu belajar pertama kali biasanya orang hanya tahu ETL jalan saja. Tapi setelah sistem menjadi besar... barulah konsep konsep ini terasa penting.

Saya jelaskan satu per satu dengan cara yang sederhana dulu, lalu sedikit masuk ke sisi teknis.

1. Slowly Changing Dimension (SCD)

Masalah klasik dalam data warehouse adalah ini:

Data dimension bisa berubah.

Contoh:

Customer pindah kota.

```
CustomerID = 101
Name       = John
City       = Jakarta
```

Beberapa bulan kemudian:

```
City = Singapore
```

Pertanyaannya:

Kalau kita melihat **sales tahun lalu**, apakah city customer harus Jakarta atau Singapore?

Ini masalah historis.

Untuk mengatasinya ada konsep **Slowly Changing Dimension**.

SCD Type 1

Update saja data lama.

```
UPDATE DimCustomer  
SET City = 'Singapore'
```

Hasil:

CustomerID	City
101	Singapore

City lama **hilang**.

Cocok untuk data yang tidak perlu history.

Contoh:

- typo correction
- email
- phone number

SCD Type 2

History disimpan.

Jadi setiap perubahan membuat **row baru**.

CustomerKey	CustomerID	City	StartDate	EndDate
1	101	Jakarta	2020	2023
2	101	Singapore	2023	NULL

Sekarang fact table tetap menunjuk ke record yang benar secara historis.

FactSales:

OrderID	CustomerKey	SalesAmount
500	1	100

Ini berarti customer masih di Jakarta saat transaksi.

Ini adalah **standar di data warehouse enterprise**.

Kimball (Data Warehouse Toolkit) menjelaskan SCD sebagai teknik penting untuk mempertahankan riwayat dimensi (Kimball & Ross, 2013).

2. Surrogate Key vs Business Key

Dimension table biasanya memiliki dua jenis key.

Contoh:

DimCustomer

Business Key

Ini key dari sistem operasional.

CustomerID = 101

Ini datang dari OLTP.

Surrogate Key

Ini key yang dibuat oleh data warehouse.

Biasanya **identity integer**.

CustomerKey = 1

Kenapa perlu?

Karena:

1. OLTP key bisa berubah
2. SCD Type 2 membutuhkan multiple rows untuk satu customer

3. join lebih cepat

Contoh:

CustomerKey	CustomerID	City
1	101	Jakarta
2	101	Singapore

Fact table akan menggunakan:

CustomerKey

bukan CustomerID.

Ini adalah **praktik standar dimensional modeling** (Kimball methodology).

3. Fact Table Grain

Ini konsep yang sangat penting tapi sering tidak dijelaskan.

Grain = level detail dari fact table.

Contoh:

FactSales bisa memiliki grain berbeda.

Grain 1

Per order.

OrderID
SalesAmount

Grain 2

Per order item.

OrderID
ProductID
Quantity

Grain 3

Per day per product.

Date
ProductID
TotalSales

Sebelum membuat fact table kita harus mendefinisikan grain.

Contoh definisi:

Grain: One row per product per order line.

Ini berarti:

OrderID + ProductID

menjadi unique.

Kalau grain tidak jelas:

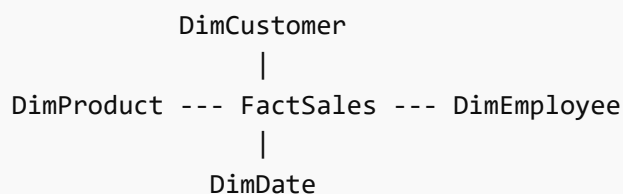
Data warehouse akan **berantakan dan sulit dianalisis**.

4. Star Schema vs Snowflake Schema

Ini dua cara menyusun dimension tables.

Star Schema

Struktur seperti bintang.



Semua dimension langsung terhubung ke fact.

Keuntungan:

- query lebih cepat
- mudah dipahami
- cocok untuk BI tools

Ini yang paling umum dipakai.

Snowflake Schema

Dimension dipecah lagi menjadi sub table.

Contoh:

```
DimProduct
  |
Category
  |
Department
```

Struktur:

```
FactSales
  |
DimProduct
  |
Category
  |
Department
```

Keuntungan:

- lebih normalised
- hemat storage

Kekurangan:

- join lebih banyak
- query lebih lambat

Karena itu **BI biasanya memakai star schema**.

5. Incremental Load vs Full Load

Ini konsep ETL yang sangat penting.

Full Load

Semua data diambil setiap ETL.

```
TRUNCATE TABLE DimCustomer

INSERT INTO DimCustomer
SELECT * FROM Customers
```

Masalah:

Kalau data besar (misalnya 200 juta row)

ETL bisa memakan waktu berjam jam.

Incremental Load

Hanya mengambil data yang berubah.

Biasanya menggunakan:

LastModified

Query:

```
SELECT *  
FROM Customers  
WHERE LastModified > @LastETLTime
```

Keuntungan:

- jauh lebih cepat
- scalable

Semua enterprise data warehouse menggunakan incremental ETL.

6. Partitioning Fact Table

Ini teknik performance untuk fact table yang sangat besar.

Bayangkan:

FactSales

memiliki:

3 Billion rows

Query seperti ini akan lambat:

```
SELECT SUM(SalesAmount)  
FROM FactSales  
WHERE OrderDate >= '2025-01-01'
```

Karena database harus scan seluruh table.

Solusinya adalah **partitioning**.

Table dibagi berdasarkan waktu.

Contoh:

```
FactSales_2023
FactSales_2024
FactSales_2025
```

Atau internal partition:

```
Partition by OrderDate
```

SQL Server akan membaca hanya partition yang relevan.

Contoh:

Query 2025 hanya membaca:

```
Partition 2025
```

Ini bisa meningkatkan performa **10x sampai 100x**.

7. Hubungan Semua Konsep Ini

Kalau digabungkan, data warehouse profesional biasanya memiliki:

```
OLTP
  ↓
Incremental ETL
  ↓
Staging Tables
  ↓
Dimension Tables (SCD Type 2)
  ↓
Fact Tables (defined grain)
  ↓
Partitioned storage
  ↓
Star Schema
  ↓
BI Tools (Tableau / Power BI)
```

8. Kenapa Ini Penting Untuk Data Analyst

Kalau kamu menggunakan Tableau atau Power BI, memahami ini membuat kamu:

1. Mengerti **kenapa query cepat atau lambat**
2. Mengerti **kenapa data historis berubah**
3. Bisa **debug ETL**
4. Bisa bekerja dengan **data engineer**

Ini biasanya skill yang membedakan:

Junior Data Analyst
vs
Senior Data Analyst / Analytics Engineer

Kalau kamu mau, saya juga bisa jelaskan **5 konsep yang bahkan lebih dalam lagi dalam data warehouse engineering**, yang sering muncul di perusahaan besar seperti Amazon, Uber, atau Netflix:

- Late Arriving Dimensions
- Degenerate Dimensions
- Bridge Tables (many-to-many dimensions)
- Snapshot Fact Tables
- Data Warehouse Slowly Changing Fact

Ini sudah masuk ke **advanced dimensional modeling**.

5 konsep yang bahkan lebih dalam lagi dalam data warehouse engineering, yang sering muncul di perusahaan besar seperti Amazon, Uber, atau Netflix:

Late Arriving Dimensions

Degenerate Dimensions

Bridge Tables (many-to-many dimensions)

Snapshot Fact Tables

Data Warehouse Slowly Changing Fact

Ini sudah masuk ke advanced dimensional modeling.

Sekarang kita masuk ke area yang biasanya baru benar benar terasa ketika data warehouse sudah **besar, kompleks, dan dipakai banyak tim**. Konsep ini muncul di perusahaan dengan data sangat besar seperti Amazon, Uber, dan Netflix.

Awalnya konsep ini terdengar rumit. Tapi kalau dilihat dari masalah nyata di sistem data, semuanya sebenarnya cukup logis.

1. Late Arriving Dimensions

Ini masalah yang sangat sering terjadi di ETL.

Biasanya alur normal seperti ini:

DimCustomer
↓
FactSales

Artinya data customer harus **sudah ada lebih dulu** sebelum sales dimasukkan.

Contoh normal:

DimCustomer		
CustomerKey	CustomerID	Name
1	C001	John

FactSales		
OrderID	CustomerKey	Amount
500	1	100

Tidak ada masalah.

Masalah: dimension datang terlambat

Kadang data transaksi masuk duluan.

Contoh:

```
OrderID = 500  
CustomerID = C001
```

Tapi record customer **belum ada di DimCustomer**.

Ini disebut **Late Arriving Dimension**.

Kalau ETL tetap memaksa join:

```
CustomerKey = NULL
```

Fact table akan rusak.

Solusi yang sering dipakai

1. Unknown Member

Buat record default di dimension.

```
CustomerKey = -1  
CustomerName = Unknown
```

Fact table:

OrderID	CustomerKey	Amount
500	-1	100

Nanti saat customer datang, ETL akan **update fact table**.

2. Inferred Dimension

ETL membuat dimension sementara.

CustomerKey	CustomerID	Name
10	C001	Unknown

Saat data lengkap datang:

```
UPDATE DimCustomer  
SET Name = 'John'
```

Ini teknik yang banyak dipakai di sistem besar.

Kimball menjelaskan ini sebagai **late arriving dimension handling**.

2. Degenerate Dimensions

Biasanya dimension adalah tabel sendiri.

Contoh:

```
DimProduct
DimCustomer
DimEmployee
```

Tetapi kadang ada atribut yang **tidak perlu dimension table**.

Contoh klasik:

```
OrderNumber
InvoiceNumber
TicketNumber
```

Ini tetap penting untuk analisis.

Tapi tidak punya atribut tambahan.

Contoh

FactSales:

```
OrderNumber
ProductKey
CustomerKey
SalesAmount
```

OrderNumber tidak punya tabel:

```
DimOrder
```

Karena tidak ada informasi tambahan.

Ini disebut **Degenerate Dimension**.

Jadi dimension tersebut **disimpan langsung di fact table**.

Ini sangat umum dalam retail dan e-commerce.

3. Bridge Tables (Many-to-Many Dimensions)

Biasanya hubungan dimension ke fact adalah 1 to many.

Contoh:

Customer → many sales
Product → many sales

Tapi kadang hubungan **many-to-many**.

Contoh nyata:

Satu produk bisa punya **banyak kategori**.

Product A
Category: Electronics
Category: Smart Home

Kalau kita simpan langsung di DimProduct, datanya tidak fleksibel.

Solusi: Bridge Table

Struktur:

DimProduct
ProductKey
ProductName

DimCategory
CategoryKey
CategoryName

BridgeProductCategory
ProductKey
CategoryKey

Fact table tetap sederhana:

FactSales
ProductKey
SalesAmount

Saat query:

```
FactSales
→ DimProduct
→ BridgeProductCategory
→ DimCategory
```

Bridge table memungkinkan **many to many relationship**.

Ini sangat umum di sistem marketplace.

4. Snapshot Fact Tables

Fact table biasanya menyimpan **event transaksi**.

Contoh:

```
OrderID
ProductID
SalesAmount
```

Ini disebut **transaction fact table**.

Tapi kadang kita ingin melihat **kondisi pada waktu tertentu**.

Contoh:

```
inventory per day
account balance per day
active users per day
```

Ini disebut **snapshot fact table**.

Contoh: inventory snapshot

Setiap hari ETL menyimpan keadaan.

```
FactInventorySnapshot
Date
ProductKey
WarehouseKey
StockLevel
```

Contoh data:

Date	Product	Stock
2025-01-01	A	100
2025-01-02	A	90
2025-01-03	A	80

Ini memungkinkan analisis:

- stock trend
- supply chain
- inventory planning

Perusahaan seperti Amazon sangat bergantung pada snapshot fact untuk warehouse analytics.

5. Slowly Changing Fact

Ini konsep yang lebih jarang dibahas.

Biasanya **fact table** tidak berubah.

Karena transaksi dianggap final.

Contoh:

OrderID
SalesAmount
OrderDate

Tetapi dalam beberapa bisnis, fakta bisa berubah.

Contoh:

Ride sharing di Uber.

Trip awal:

TripID
Fare = 20

Kemudian:

Fare adjustment
Fare = 18

Atau refund.

Cara menangani

Ada beberapa pendekatan.

1. Update fact

```
UPDATE FactTrip
SET Fare = 18
```

2. Adjustment record

Tambahkan record baru.

```
TripID Amount Type
1001    20    Original
1001    -2    Adjustment
```

Total:

```
SUM(Amount) = 18
```

Ini sering digunakan dalam finance systems.

6. Bagaimana Semua Ini Digunakan Bersama

Sistem data warehouse besar biasanya terlihat seperti ini:

```
OLTP Systems
  ↓
Streaming / Batch ETL
  ↓
Staging Tables
  ↓
Dimension Tables (SCD)
  ↓
Bridge Tables
  ↓
Fact Tables
  ↓
Snapshot Facts
  ↓
BI Layer (Tableau / Power BI)
```

Tools seperti Tableau atau Microsoft Power BI membaca model ini untuk membuat dashboard.

7. Insight yang Biasanya Baru Dipahami Setelah Bekerja

Beberapa realisasi yang biasanya datang setelah pengalaman:

1. **Fact table design lebih penting daripada dashboard**
2. ETL sering lebih kompleks daripada analisis
3. Dimension modeling menentukan kualitas analytics
4. Data warehouse bukan sekadar database besar
5. BI tool hanya layer paling atas

Kalau kamu mau, saya juga bisa jelaskan **arsitektur data modern yang dipakai perusahaan besar sekarang**, misalnya:

- Data Lake vs Data Warehouse
- Lakehouse architecture
- Medallion Architecture (Bronze, Silver, Gold)
- Modern stack seperti **Snowflake, dbt, Airflow**

Ini biasanya yang dipakai oleh tim data modern di perusahaan teknologi besar.